



Faculty of Engineering
Software Engineering
Laboratory Manual

NAME:

ENROLLMENT NUMBER:

SUBJECT NAME:

SUBJECT CODE: **BRANCH:**

SEMESTER: **ACADEMIC YEAR:**



Swaminarayan university
Publication

ISBN Number:



सर्व विद्या अधिपतये

Certificate

This is to certify that Mr. / Ms. / Mrs.
having enrollment number of Semester ME /
BE / DIPLOMA in.....has satisfactorily completed his
term working (Subject name) (subject code:.....)
during academic year at Faculty of Engineering, Swaminarayan University.

Place:

Date:

ASHWINI WALAKE
SUBJECT IN-CHARGE

HOD

DR. BHAVNA SONI
DEAN

List of Practical

Sr. No.	Title	Page No.	Date	Sign
1	Study the complete Software Development Life Cycle (SDLC) and analyze various activities conducted as a part of various phases. For each SDLC phase, identify the objectives and summaries outcomes.			
2	Consider any project to be developed in any technology as a Software Architect or Project Manager. Construct Software Requirement Specification (SRS) document for the project.			
3	To prepare a Software Requirement Specification (SRS) for the Hindi Film Database Management System and estimate the software development effort, schedule, team size, and cost using Basic and Intermediate COCOMO models.			
4	To study the Function Point Analysis (FPA) technique, calculate the Function Points (FP) for a software project, identify errors in the given case study, and estimate the software development effort.			
5	To design the Control Flow Graph (CFG) for a given Java program, calculate its Cyclomatic Complexity, and identify the independent execution paths required for Basis Path Testing			
6	To prepare the software documents for a Library Information System by developing the Software Requirement Specification (SRS), UML diagrams, Data Flow Diagrams (DFD), Entity Relationship (ER) Diagram, and Test Cases.			
7	Study any two Open Source DevOps tools used for Infrastructure Automation, Configuration Management, Deployment Automation, Performance Management, Log Management, or Monitoring.			

Practical-1

Aim: Study the complete Software Development Life Cycle (SDLC) and analyze various activities conducted as a part of various phases. For each SDLC phase, identify the objectives and summaries outcomes.

Objectives: After completing this experiment, students will be able to

1. Understand the concept of Software Development Life Cycle.
2. Identify different phases of SDLC.
3. Analyze the activities performed during each phase.
4. Understand the importance of documentation.
5. Compare different SDLC models.
6. Select an appropriate SDLC model for different software projects.

Introduction:

Software Development Life Cycle (SDLC) is a systematic process followed for developing high-quality software that meets customer requirements. It provides a structured framework for planning, designing, developing, testing, deploying, and maintaining software systems.

The primary objective of SDLC is to produce high-quality software within the specified budget and time while minimizing risks and ensuring customer satisfaction.

A well-defined SDLC helps organizations improve software quality, reduce development cost, increase productivity, and ensure timely project completion.

Why is SDLC Important?

Following an SDLC model provides several advantages:

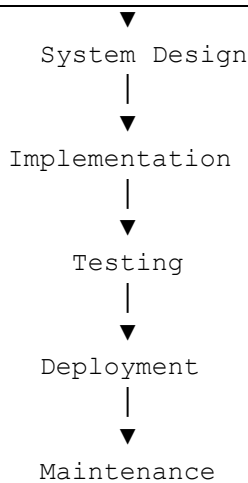
- Provides a structured approach to software development.
- Improves project planning and management.
- Ensures customer requirements are properly understood.
- Reduces project risks.
- Helps detect errors at an early stage.
- Improves software quality.
- Facilitates better communication among stakeholders.
- Reduces maintenance costs.
- Enables proper documentation throughout the project lifecycle.

Generic Software Development Life Cycle

Requirement Analysis



Feasibility Study



Phases of SDLC

1. Requirement Analysis

Description

Requirement Analysis is the first and one of the most important phases of SDLC. During this phase, the software development team communicates with the client to understand the project requirements.

The collected requirements are analyzed, validated, documented, and finalized.

Activities

- Conduct meetings with stakeholders.
- Gather functional requirements.
- Gather non-functional requirements.
- Analyze business processes.
- Identify project scope.
- Prepare Software Requirement Specification (SRS).

2. Feasibility Study

Description

Before investing time and resources, the project team performs a feasibility study to determine whether the proposed project is practical and economically viable.

Types of Feasibility

Technical Feasibility

Checks whether the required technology, hardware, and software are available.

Economic Feasibility

Determines whether the project is financially beneficial.

Operational Feasibility

Checks whether users can effectively use the proposed system.

Legal Feasibility

Ensures compliance with legal and regulatory requirements.

Schedule Feasibility

Determines whether the project can be completed within the specified timeline.

3. System Design

Description

After requirements are finalized, system architects design the complete software architecture.

Design specifies how the software will work internally.

Types of Design

High-Level Design (HLD)

Includes:

- Architecture
- Modules
- Data Flow
- Interfaces

Low-Level Design (LLD)

Includes:

- Algorithms
- Database tables
- Class diagrams
- Program logic

Activities

- Design architecture
- Design database
- Prepare UML diagrams
- Design user interface
- Define APIs

4. Implementation (Coding)

Description

Implementation is the phase where developers write source code based on the design documents. Coding standards and best practices are followed to produce maintainable and efficient software.

Activities

- Coding
- Unit Testing
- Code Review
- Version Control

5. Testing

Description

Testing ensures that the developed software functions correctly and satisfies customer requirements. The objective is to identify and remove defects before deployment.

Activities

- Test Planning
- Test Case Preparation
- Bug Reporting
- Bug Fixing

6. Deployment

Description

After successful testing, the software is installed in the client's environment for actual use.

Activities

- Software Installation
- Configuration
- User Training
- Data Migration

7. Maintenance

Description

Maintenance begins after deployment and continues throughout the software's operational life. It includes fixing bugs, improving performance, and adding new features.

Summary of SDLC Phases

Phase	Objective	Major Activities	Deliverables
Requirement Analysis	Understand customer needs	Requirement gathering, analysis	SRS Document
Feasibility Study	Evaluate project viability	Technical, economic, operational analysis	Feasibility Report
System Design	Create technical blueprint	Architecture, database, UI, UML	Design Document
Implementation	Develop software	Coding, unit testing	Source Code
Testing	Verify software quality	Functional and non-functional testing	Test Reports
Deployment	Deliver software	Installation, configuration	Deployed System
Maintenance	Improve and support software	Bug fixes, enhancements	Updated Software

Applications

SDLC is used in the development of:

- Banking Systems
- Hospital Management Systems
- Library Management Systems
- E-commerce Websites
- Airline Reservation Systems
- Government Portals
- Mobile Applications
- Cloud-Based Applications
- Embedded Systems
- Enterprise Resource Planning (ERP) Systems

Result

The Software Development Life Cycle (SDLC) was studied in detail.

Practical-2

Aim: Consider any project to be developed in any technology as a Software Architect or Project Manager. Construct Software Requirement Specification (SRS) document for the project.

Objectives

After completing this experiment, students will be able to:

1. Understand the role of a Software Architect and Project Manager.
2. Identify project objectives and scope.
3. Conduct a feasibility study.
4. Prepare project planning documents.
5. Identify project risks.
6. Create a Work Breakdown Structure (WBS).
7. Prepare a project schedule using a Gantt Chart.
8. Allocate resources and define team responsibilities.

Introduction

Software project planning is one of the most important activities in software engineering. Before any coding begins, the project must be planned carefully to ensure that it is completed on time, within budget, and according to customer requirements.

A Software Architect focuses on the technical design of the system, including software architecture, technology selection, database design, and system integration. A Project Manager is responsible for planning, scheduling, budgeting, team management, communication, and risk management.

Proper planning minimizes project failures and improves software quality.

Roles and Responsibilities

Software Architect

A Software Architect is responsible for designing the overall structure of the software system.

Responsibilities

- Understand business requirements.
- Select suitable technologies.
- Design system architecture.
- Prepare UML diagrams.
- Design database structure.
- Ensure software scalability.
- Ensure security and performance.
- Guide developers during implementation.

Project Manager

A Project Manager ensures the successful completion of the project.

Responsibilities

- Project planning
 - Team management
 - Resource allocation
 - Cost estimation
-

-
- Risk management
 - Schedule preparation
 - Client communication
 - Progress monitoring
 - Quality assurance

Selected Project

Online Library Management System

Project Description

The Online Library Management System is a web-based application that automates the management of library operations. It helps librarians manage books, students, issue and return transactions, fines, and reports efficiently.

The system replaces manual record-keeping and improves accuracy, accessibility, and operational efficiency.

Problem Statement

Traditional library systems rely on manual registers and paperwork, which can lead to errors, delays, and difficulty in maintaining records. Students often face challenges in searching for books and checking their availability.

The proposed Online Library Management System aims to provide a centralized, digital platform for managing library resources.

Project Objectives

- Automate library operations.
- Maintain digital records of books.
- Manage student registrations.
- Track book issue and return.
- Calculate fines automatically.
- Generate reports.
- Reduce paperwork.
- Improve user experience.

Scope of the Project

The system includes:

- User Login
- Student Registration
- Book Management
- Book Search
- Book Issue
- Book Return
- Fine Calculation
- Report Generation
- Admin Dashboard

Out of Scope

- Online payment gateway
- E-book reading
- Mobile application

-
- Barcode integration (future enhancement)

Stakeholders

Stakeholder	Role
Library Administrator	Manages the system
Librarian	Maintains books and transactions
Students	Search and borrow books
College Management	Monitors reports
Software Development Team	Develops the system

Functional Requirements

The system shall:

1. Allow user login.
2. Add, edit, and delete books.
3. Register students.
4. Search books by title, author, or category.
5. Issue books.
6. Return books.
7. Calculate fines.
8. Generate daily and monthly reports.
9. Maintain transaction history.
10. Manage user accounts.

Non-Functional Requirements

- User-friendly interface
- High performance
- Secure authentication
- Data backup
- Scalability
- Reliability
- Availability
- Easy maintenance

Technology Stack

Component	Technology
Frontend	HTML5, CSS3, JavaScript
Backend	Java (Spring Boot) / PHP / Python (Django)
Database	MySQL
IDE	IntelliJ IDEA / Eclipse / VS Code
Server	Apache Tomcat
Version Control	Git
Operating System	Windows/Linux

Feasibility Study

A feasibility study evaluates whether the project is practical and worthwhile.

1. Technical Feasibility

Analysis

- Required technologies are readily available.
- Development tools are open-source or commonly used.
- Team members have the required technical skills.

Conclusion

Technically Feasible

2. Economic Feasibility

Estimated Cost

Item	Cost (₹)
Software Development	2,50,000
Testing	40,000
Documentation	20,000
Hardware	60,000
Training	30,000
Total	4,00,000

Expected Benefits

- Reduced paperwork
- Faster operations
- Better record management
- Lower administrative costs
- Improved productivity

Conclusion

Economically Feasible

3. Operational Feasibility

Analysis

- Easy to use.
- Minimal training required.
- Supports daily library operations.
- Accepted by staff and students.

Conclusion

Operationally Feasible

4. Legal Feasibility

The system complies with:

- Software licensing policies
 - Institutional data privacy guidelines
-

- Copyright regulations for digital records

Conclusion

Legally Feasible

5. Schedule Feasibility

Estimated project duration: **16 weeks (4 months)**

The project can be completed within the proposed timeline.

Conclusion

Schedule Feasible

Team Structure

Role	Responsibilities
Project Manager	Planning, scheduling, monitoring
Software Architect	System architecture and technology selection
Business Analyst	Requirement gathering
UI/UX Designer	Interface design
Backend Developer	Business logic implementation
Frontend Developer	User interface development
Database Administrator	Database design and maintenance
Tester	Software testing and quality assurance
Technical Writer	Documentation

Resource Requirements

Hardware

- Intel Core i5 or above
- 8 GB RAM
- 500 GB HDD/SSD
- Broadband Internet

Software

- Windows 10 / Linux
- Java JDK
- MySQL
- Apache Tomcat
- Visual Studio Code
- Git
- Chrome Browser

Project Schedule (Gantt Chart)

Activity	Week 1–2	Week 3–4	Week 5–6	Week 7–10	Week 11–13	Week 14–15	Week 16
Requirement Gathering	✓						
Feasibility Study	✓	✓					

Activity	Week 1–2	Week 3–4	Week 5–6	Week 7–10	Week 11–13	Week 14–15	Week 16
SRS Preparation		✓	✓				
System Design			✓	✓			
Coding				✓	✓		
Testing					✓	✓	
Deployment						✓	
Maintenance							✓

Risk Analysis

Risk	Probability	Impact	Mitigation Strategy
Changing Requirements	Medium	High	Frequent client meetings
Schedule Delay	Medium	High	Weekly progress reviews
Team Member Unavailability	Low	Medium	Backup resource planning
Technology Issues	Medium	Medium	Prototype and early testing
Data Loss	Low	High	Regular backups
Security Vulnerabilities	Medium	High	Security testing and code reviews

Project Constraints

- Limited budget.
- Fixed project deadline.
- Limited development team.
- Internet connectivity required.
- Availability of client for feedback.

Success Criteria

The project will be considered successful if:

- All functional requirements are implemented.
- The system is delivered on time.
- The project remains within budget.
- User acceptance testing is successful.
- The system operates reliably with minimal defects.

Advantages of Proper Project Planning

- Better resource utilization.
- Improved communication.
- Reduced project risks.
- Clear project objectives.
- Better time management.
- Increased customer satisfaction.
- Easier monitoring and control.
- Improved software quality.

Applications

Project planning techniques are widely used in:

- Banking Systems
- Hospital Management Systems
- E-commerce Platforms
- Airline Reservation Systems
- ERP Solutions
- Library Management Systems
- Government Portals
- Educational Institutions

Result

A software project was successfully planned by defining its objectives, scope, feasibility, resources, risks, schedule, and team responsibilities. The exercise demonstrated the importance of systematic project planning before software development begins.

Practical-3

Aim: To prepare a Software Requirement Specification (SRS) for the Hindi Film Database Management System and estimate the software development effort, schedule, team size, and cost using Basic and Intermediate COCOMO models.

Construct Software Requirement Specification (SRS) document for the project.

Considering your immense expertise in software development, The Absolute Beginners Inc. has recently allotted you a mega project. The goal of the project is to create a database of all Hindi films released since 2000. The software would allow one to generate a list of top ten hit films, top ten flop films, best comedy films, and so on. Using your prior experience you have decided the approximate sizes of each module of the software as follow:

- Data entry (0.9 KDSI)
- Data update (0.7 KDSI)
- Query (0.9 KDSI)
- Report generation and display (2 KDSI)
- Storage constraints (Low)
- Experience in developing similar software (High)
- Programming capabilities of the developers (High)
- Application of software engineering methods (High)
- Use of software tools (High)

Solve the problem by Applying Basic and intermediate COCOMO

Find Project Type? Find Project Size?

- Find Initial Effort Estimation? o Find Adjusted Effort Estimation? o Find schedule? o Find minimum size of the team you would require to develop this system?
- Assuming that your client would pay Rs. 50,000 per month of development, how much would be the likely billing?

1. Software Requirement Specification (SRS)

1.1 Introduction

Project Name

Hindi Film Database Management System

Purpose

The purpose of this project is to develop a software system that maintains a database of all Hindi films released since the year 2000. The software enables users to search, update, and generate reports such as Top 10 Hit Movies, Top 10 Flop Movies, Best Comedy Movies, Best Action Movies, Highest Grossing Movies, etc.

1.2 Scope

The software will:

- Store Hindi movie information.
- Add new movie records.
- Update movie details.
- Search movie records.
- Generate different reports.
- Display movie rankings.
- Maintain a centralized database.

1.3 Objectives

- Maintain movie database digitally.
- Reduce manual record keeping.
- Provide quick searching.
- Generate reports automatically.
- Improve data accuracy.

2. Functional Requirements

The system shall provide the following functions:

FR1: User Login

- Admin login authentication.

FR2: Data Entry

- Add movie details.
- Add actors and directors.
- Add release year and genre.
- Add box office collection.

FR3: Data Update

- Modify movie information.
- Delete incorrect records.
- Update box office collection.

FR4: Query Processing

- Search by movie name.
- Search by actor.
- Search by director.
- Search by genre.
- Search by release year.

FR5: Report Generation

- Top 10 Hit Movies
 - Top 10 Flop Movies
 - Best Comedy Movies
 - Best Action Movies
 - Highest Rated Movies
 - Movies Released in a Particular Year
-

3. Non-Functional Requirements

- User-friendly interface
 - High performance
 - Secure login
 - Reliable system
 - Easy maintenance
 - Fast searching
 - Database backup
-

4. Hardware Requirements

- Intel Core i3 Processor
 - 4 GB RAM
 - 500 GB Hard Disk
-

5. Software Requirements

- Windows/Linux
 - MySQL Database
 - Java/Python/PHP
 - Visual Studio Code / Eclipse
-

7. Module Size

Module	Size (KDSI)
Data Entry	0.9
Data Update	0.7
Query	0.9
Report Generation	2.0
Total Size	4.5 KDSI

8. COCOMO Estimation

Step 1: Project Type

The project is:

- Small in size
- Familiar application
- Experienced developers
- Flexible requirements

Project Type = Organic

Step 2: Project Size

Total Size = $0.9 + 0.7 + 0.9 + 2.0$
= **4.5 KDSI**

9. Basic COCOMO Estimation

Formula

$$\text{Effort (E)} = 2.4 \times (\text{KDSI})^{1.05}$$

Substitute the values:

$$E = 2.4 \times (4.5)^{1.05}$$

$$E = 2.4 \times 4.85$$

$$\text{Effort} = 11.64 \text{ Person-Months}$$

Development Schedule

Formula

$$T = 2.5 \times (\text{Effort})^{0.38}$$

$$T = 2.5 \times (11.64)^{0.38}$$

$$T = 6.34 \text{ Months}$$

Team Size

Formula

$$\text{Team Size} = \text{Effort} / \text{Schedule}$$

$$= 11.64 / 6.34$$

$$= 1.84 \approx 2 \text{ Members}$$

10. Intermediate COCOMO

Cost Drivers

Cost Driver	Rating	Multiplier
Storage Constraints	Low	0.94
Experience in Similar Software	High	0.91
Programmer Capability	High	0.86
Software Engineering Methods	High	0.91
Software Tools	High	0.91

Effort Adjustment Factor (EAF)

$$\text{EAF} = 0.94 \times 0.91 \times 0.86 \times 0.91 \times 0.91$$

$$\approx 0.61$$

Adjusted Effort

Formula

$$\text{Adjusted Effort} = \text{Basic Effort} \times \text{EAF}$$

$$= 11.64 \times 0.61$$

$$= 7.10 \text{ Person-Months}$$

Adjusted Development Time

Formula

$$T = 2.5 \times (7.10)^{0.38}$$

$$= 5.3 \text{ Months}$$

Team Size

$$= 7.10 / 5.3$$

$$= 1.34 \approx 2 \text{ Members}$$

11. Project Cost

Client Payment= **₹50,000 per Person-Month**

$$\text{Total Billing} = 7.10 \times 50,000 = \mathbf{₹3,55,000}$$

12. Final Results

Parameter	Result
Project Type	Organic
Project Size	4.5 KDSI
Initial Effort (Basic COCOMO)	11.64 Person-Months
Adjusted Effort (Intermediate COCOMO)	7.10 Person-Months
Development Schedule	5.3 Months
Minimum Team Size	2 Members
Estimated Billing	₹3,55,000

14. Result

The **Software Requirement Specification (SRS)** for the Hindi Film Database Management System was prepared successfully. Using **Basic and Intermediate COCOMO**, the project effort, schedule, team size, and development cost were estimated.

Practical-4

AIM: To study the Function Point Analysis (FPA) technique, calculate the Function Points (FP) for a software project, identify errors in the given case study, and estimate the software development effort.

Objective

- To understand Function Point Analysis.
- To calculate Unadjusted Function Points (UFP).
- To calculate Value Adjustment Factor (VAF).
- To estimate the final Function Points.
- To estimate software effort using Function Points.

Function Point Analysis (FPA) is a software estimation technique used to measure the size of a software.

Components of Function Point Analysis

1. **External Inputs (EI)** – User inputs to the system.
2. **External Outputs (EO)** – Reports or outputs generated.
3. **External Inquiries (EQ)** – User queries without updating data.
4. **Internal Logical Files (ILF)** – Files maintained by the system.
5. **External Interface Files (EIF)** – Files used but maintained by another system.

Case Study

Consider an **Online Library Management System** with the following details:

Function Type	Count	Complexity	Weight
External Inputs (EI)	10	Average	4
External Outputs (EO)	8	Average	5
External Inquiries (EQ)	5	Average	4
Internal Logical Files (ILF)	4	Average	10
External Interface Files (EIF)	2	Average	7

Step 1: Calculate Unadjusted Function Points (UFP)

Function Type Count Weight Total

EI	10	4	40
EO	8	5	40
EQ	5	4	20
ILF	4	10	40
EIF	2	7	14

Total UFP 154

Unadjusted Function Points (UFP) = 154

Step 2: Value Adjustment Factor (VAF)

The Value Adjustment Factor is calculated using the **14 General System Characteristics (GSCs)** such as:

- Data Communications
 - Distributed Processing
 - Performance
 - Heavily Used Configuration
 - Transaction Rate
-

-
- Online Data Entry
 - End User Efficiency
 - Online Update
 - Complex Processing
 - Reusability
 - Installation Ease
 - Operational Ease
 - Multiple Sites
 - Ease of Change

Assume the **Total Degree of Influence (TDI) = 35**

Formula:

$$\text{VAF} = 0.65 + (0.01 \times \text{TDI})$$

$$\text{VAF} = 0.65 + (0.01 \times 35)$$

$$\text{VAF} = 1.00$$

Step 3: Calculate Function Points (FP)

Formula:

$$\text{FP} = \text{UFP} \times \text{VAF}$$

$$\text{FP} = 154 \times 1.00$$

$$\text{FP} = 154$$

Step 4: Effort Estimation

Assume:

Productivity = 10 Function Points per Person-Month

Formula:

$$\text{Effort} = \text{FP} / \text{Productivity}$$

$$\text{Effort} = 154 / 10$$

$$\text{Effort} = 15.4 \text{ Person-Months}$$

Error Identification in Case Study

Error

The original calculation ignored the **Value Adjustment Factor (VAF)** and considered only the Unadjusted Function Points (UFP), leading to an inaccurate estimation.

Correct Approach

- Calculate UFP.
- Calculate TDI.
- Determine VAF.
- Multiply UFP by VAF to obtain the final Function Points.
- Estimate effort using the corrected FP value.

Advantages of Function Point Analysis

- Independent of programming language.
- Can be applied early in the software development lifecycle.
- Provides accurate size estimation.
- Useful for project planning and cost estimation.
- Easy to compare different software projects.

Limitations

- Depends on accurate requirement analysis.
 - Time-consuming for large systems.
 - Requires trained analysts.
-

-
- Complexity ratings may vary between evaluators.

Applications

- Software effort estimation
- Cost estimation
- Project planning
- Productivity measurement
- Software project comparison

Result

The Function Point Analysis for the Online Library Management System was completed successfully. The Unadjusted Function Points (UFP), Value Adjustment Factor (VAF), Final Function Points (FP), and estimated effort were calculated.

Final Calculation Summary

Parameter	Value
Unadjusted Function Points (UFP)	154
Total Degree of Influence (TDI)	35
Value Adjustment Factor (VAF)	1.00
Final Function Points (FP)	154
Productivity	10 FP/Person-Month
Estimated Effort	15.4 Person-Months

Practical-5

Aim: To design the Control Flow Graph (CFG) for a given Java program, calculate its Cyclomatic Complexity, and identify the independent execution paths required for Basis Path Testing.

Consider the following Java code segment:

```
public Hashtable countAlphabet(String aString){
    Hashtable table = new Hashtable();
    If (aString.length > 4000) return table;
    StringBuffer buffer = new StringBuffer(aString);
    while (buffer.length() > 0){
        String firstChar = buffer.substring(0, 1);
        Integer count = (Integer)table.get(firstChar);
        if (count == null){
            count = new Integer(1);
        } else{
            count = new Integer(count.intValue() + 1);
        }
        table.put(firstChar, count);
        buffer.delete(0, 1);
    }
    return table;
}
```

1. Guarantees that all independent execution path is exercised at least once
2. Guarantees that both the true and false side of all logical decisions are exercised;
3. Executes the loop at the boundary values and within the boundaries.

Sketch out Design control flow diagram and Apply Cyclomatic complexity for given Code. Identify numbers of Independence path require for testing.

Objectives

- To understand White Box Testing.
- To draw the Control Flow Graph (CFG).
- To calculate Cyclomatic Complexity.
- To identify independent paths.
- To prepare test cases for basis path testing.

Theory

Cyclomatic Complexity is a software metric developed by **Thomas J. McCabe**. It measures the complexity of a program by counting the number of linearly independent execution paths through the source code. It is mainly used in **White Box Testing** to determine the minimum number of test cases required to achieve complete branch coverage.

Advantages

- Measures program complexity.
- Helps identify complex modules.
- Determines the minimum number of test cases.
- Improves software quality.
- Supports basis path testing.

Given Java Program

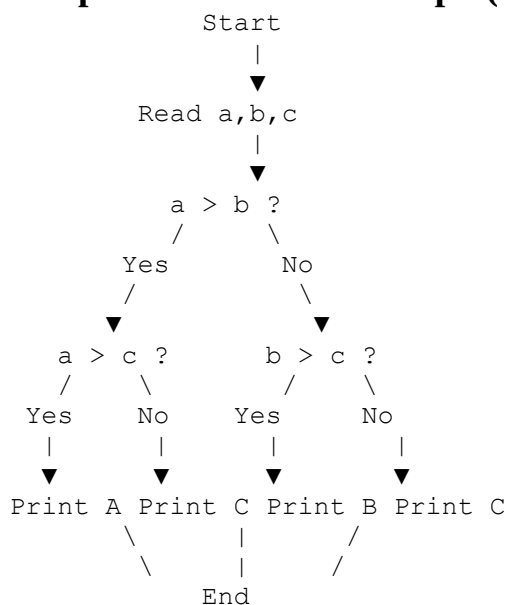
```
public class LargestNumber {

    public static void main(String args[]) {

        int a = 10;
        int b = 20;
        int c = 15;

        if(a > b) {
            if(a > c)
                System.out.println("A is Largest");
            else
                System.out.println("C is Largest");
        }
        else {
            if(b > c)
                System.out.println("B is Largest");
            else
                System.out.println("C is Largest");
        }
    }
}
```

Step 1: Control Flow Graph (CFG)



Step 2: Number of Decision Nodes

The program contains **3 decision statements**:

1. `if(a > b)`
2. `if(a > c)`
3. `if(b > c)`

Decision Nodes = 3

Step 3: Cyclomatic Complexity

Method 1

Formula:

$$[\\ V(G) = \text{Decision Nodes} + 1 \\]$$

$$[\\ V(G) = 3 + 1 = 4 \\]$$

Cyclomatic Complexity = 4

Method 2

Using the graph formula:

$$[\\ V(G) = E - N + 2 \\]$$

Where:

- **E (Edges) = 11**
- **N (Nodes) = 9**

$$[\\ V(G) = 11 - 9 + 2 = 4 \\]$$

Cyclomatic Complexity = 4

Step 4: Independent Paths

The number of independent paths is equal to the Cyclomatic Complexity.

Therefore,

Independent Paths = 4

Path 1

Start $\rightarrow a > b$ (True) $\rightarrow a > c$ (True) $\rightarrow$ Print A $\rightarrow$ End

Path 2

Start $\rightarrow a > b$ (True) $\rightarrow a > c$ (False) $\rightarrow$ Print C $\rightarrow$ End

Path 3

Start $\rightarrow a > b$ (False) $\rightarrow b > c$ (True) $\rightarrow$ Print B $\rightarrow$ End

Path 4

Start $\rightarrow a > b$ (False) $\rightarrow b > c$ (False) $\rightarrow$ Print C $\rightarrow$ End

Step 5: Test Cases

Test Case Input (a,b,c) Expected Output Path Covered

TC1	30,20,10	A is Largest	Path 1
TC2	30,20,40	C is Largest	Path 2
TC3	10,30,20	B is Largest	Path 3
TC4	10,20,30	C is Largest	Path 4

Boundary Testing

Boundary testing checks values near the limits.

Examples:

- $a = b$
 - $b = c$
 - $a = c$
 - $a = 0$
 - Negative values
 - Maximum integer values
-

Applications

Cyclomatic Complexity is used in:

- White Box Testing
-

-
- Unit Testing
 - Software Quality Measurement
 - Code Review
 - Risk Analysis
 - Test Case Design
-

Advantages

- Determines minimum test cases.
 - Detects complex code.
 - Improves code maintainability.
 - Supports complete branch coverage.
 - Helps reduce software defects.
-

Limitations

- Does not measure code quality directly.
 - High complexity does not always mean poor software.
 - Difficult to apply to very large systems.
-

Result

The **Control Flow Graph (CFG)** was successfully designed for the given Java program. The **Cyclomatic Complexity** was calculated as **4**, indicating that **4 independent execution paths** and a minimum of **4 test cases** are required for complete basis path testing.

Practical-6

Aim: To prepare the software documents for a Library Information System by developing the Software Requirement Specification (SRS), UML diagrams, Data Flow Diagrams (DFD), Entity Relationship (ER) Diagram, and Test Cases.

Objectives

- To understand software documentation.
 - To prepare SRS for the project.
 - To design UML diagrams.
 - To prepare DFD and ER diagrams.
 - To design test cases.
 - To understand software project development.
-

1. Project Introduction

The **Library Information System (LIS)** is a software application used to manage library operations such as maintaining book records, member registration, book issue/return, fine calculation, and report generation. It replaces manual record keeping with an automated system.

2. Objectives

- Automate library operations.
 - Maintain book records.
 - Manage member details.
 - Issue and return books.
 - Calculate fines automatically.
 - Generate reports.
-

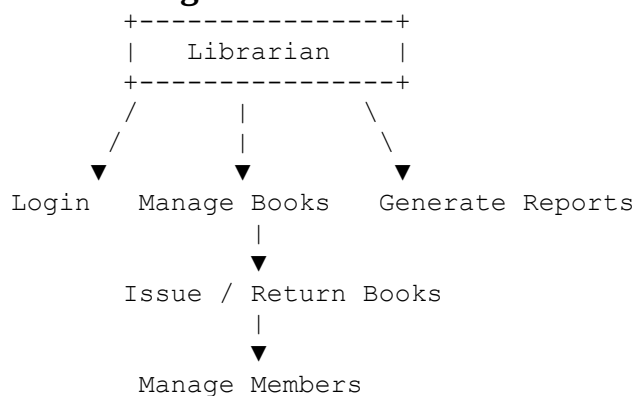
3. Functional Requirements

- User Login
 - Add/Edit/Delete Books
 - Register Members
 - Search Books
 - Issue Books
 - Return Books
 - Fine Calculation
 - Generate Reports
-
-

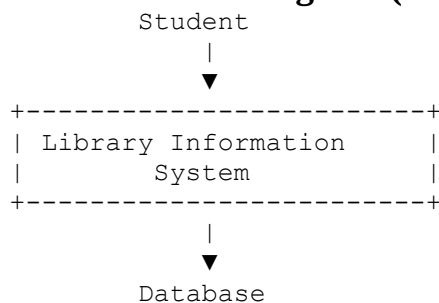
4. Non-Functional Requirements

- User-friendly interface
- Secure login
- Fast response
- Reliable database
- Data backup
- Easy maintenance

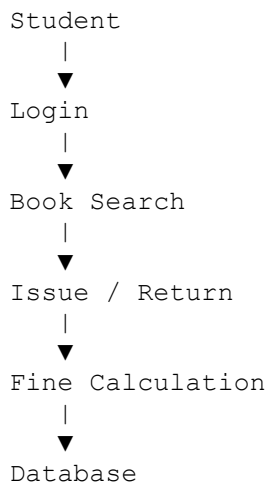
5. Use Case Diagram



6. Data Flow Diagram (Level 0)



7. Data Flow Diagram (Level 1)



8. Entity Relationship (ER) Diagram

BOOK

Book_ID
Title
Author
Publisher

|
| Issues
|

MEMBER

Member_ID
Name
Address
Phone

|
| Generates
|

TRANSACTION

Transaction_ID
Issue_Date
Return_Date
Fine

9. Database Tables

Book Table

Field	Type
Book_ID	Integer
Title	Varchar
Author	Varchar
Publisher	Varchar
Category	Varchar

Member Table

Field	Type
Member_ID	Integer
Name	Varchar
Address	Varchar
Phone	Varchar

Transaction Table

Field	Type
Transaction_ID	Integer
Book_ID	Integer

Field	Type
Member_ID	Integer
Issue_Date	Date
Return_Date	Date
Fine	Decimal

10. Test Cases

Test Case ID	Test Scenario	Expected Result
TC01	Login with valid credentials	Login Successful
TC02	Login with invalid password	Error Message
TC03	Add New Book	Book Added Successfully
TC04	Search Existing Book	Book Displayed
TC05	Issue Book	Book Issued Successfully
TC06	Return Book	Fine Calculated
TC07	Generate Report	Report Generated

11. Project Modules

- Login Module
 - Book Management Module
 - Member Management Module
 - Issue & Return Module
 - Fine Calculation Module
 - Report Generation Module
-

12. Advantages

- Reduces paperwork.
 - Fast book searching.
 - Automatic fine calculation.
 - Accurate record management.
 - Improves efficiency.
-

13. Applications

- Schools
 - Colleges
 - Universities
 - Public Libraries
 - Digital Libraries
-
-

14. Result

The documentation for the **Library Information System** was successfully prepared, including SRS, Use Case Diagram, DFD, ER Diagram, database design, and test cases.

Practical-7

Aim: Study any two Open Source DevOps tools used for Infrastructure Automation, Configuration Management, Deployment Automation, Performance Management, Log Management, or Monitoring.

Selected Tools:

1. Docker
2. Chef

Objectives

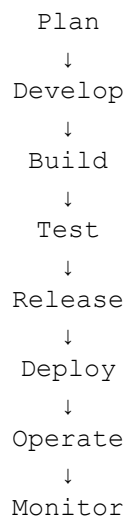
- To understand the concept of DevOps.
- To study Docker and Chef.
- To understand containerization and configuration management.
- To learn the applications of DevOps tools in software development.

Introduction

What is DevOps?

DevOps is a software development methodology that combines **Development (Dev)** and **Operations (Ops)**. It promotes collaboration between developers and operations teams to automate software development, testing, deployment, and monitoring.

DevOps Lifecycle

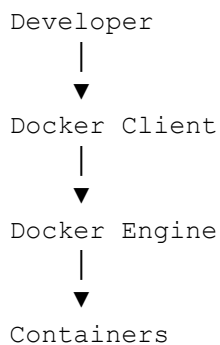


Tool 1: Docker

Introduction

Docker is an open-source containerization platform that allows developers to package an application along with its dependencies into lightweight containers. Containers ensure that applications run consistently across different environments.

Docker Architecture



Components of Docker

- Docker Client
- Docker Engine
- Docker Images
- Docker Containers
- Docker Hub
- Docker Registry

Features

- Lightweight containers
- Fast deployment
- Platform independence
- Easy application scaling
- Resource efficient
- High portability

Common Docker Commands

Command	Description
<code>docker --version</code>	Check Docker version
<code>docker images</code>	Display available images
<code>docker pull ubuntu</code>	Download Ubuntu image
<code>docker run ubuntu</code>	Run a container
<code>docker ps</code>	Show running containers
<code>docker stop <container_id></code>	Stop a container

Advantages

- Faster deployment
- Consistent execution
- Easy scalability
- Efficient resource utilization
- Simplified application packaging

Applications

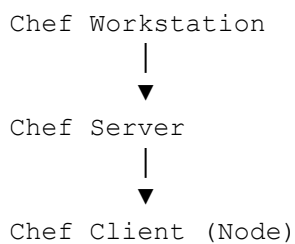
- Cloud Computing
- Microservices
- Continuous Integration/Continuous Deployment (CI/CD)
- Software Testing
- Web Application Deployment

Tool 2: Chef

Introduction

Chef is an open-source **Configuration Management** tool used to automate the configuration, deployment, and management of servers and infrastructure using code.

Chef Architecture



Components

- Chef Workstation
- Chef Server
- Chef Client
- Cookbooks
- Recipes
- Nodes

Features

- Infrastructure as Code (IaC)
- Automated configuration
- Centralized server management
- Reusable recipes
- Scalable infrastructure

Advantages

- Reduces manual configuration
-

-
- Faster server deployment
 - Consistent environments
 - Easy maintenance
 - Supports cloud platforms
-

Applications

- Server Configuration
 - Cloud Infrastructure
 - Data Center Automation
 - Continuous Deployment
 - Configuration Management
-

Comparison of Docker and Chef

Feature	Docker	Chef
Purpose	Containerization	Configuration Management
Main Function	Package and run applications	Configure and manage servers
Automation	Application Deployment	Infrastructure Automation
Uses Containers	Yes	No
Platform Support	Cross-platform	Cross-platform
Primary Users	Developers	System Administrators / DevOps Engineers

Advantages of DevOps

- Faster software delivery
 - Improved collaboration
 - Continuous Integration (CI)
 - Continuous Deployment (CD)
 - Reduced deployment failures
 - Better software quality
 - Faster issue resolution
 - Automated workflows
-

Limitations

- Initial setup cost
 - Learning curve
 - Requires skilled professionals
 - Complex for small projects
-

Applications of DevOps

- Web Applications
- Mobile Applications
- Cloud Computing
- Banking Systems
- E-commerce Platforms
- Enterprise Applications
- Software Product Companies

Result

The study of two open-source DevOps tools, **Docker** and **Chef**, was completed successfully. Their architecture, features, commands, advantages, and applications were understood, highlighting their importance in modern software development and deployment.